SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Compliance & Secure Coding Practices
Check List 02

Project: A Lightweight Detect, Defense, and User Awareness System for AI-Driven Scams

Project information	Details
Project ID	R26-CS-011
Student ID	IT22277268
Student name	Aberathna H.M.M.K
Student ID	IT22299048
Student name	Vidara S V
Student ID	IT22628718
Student name	Priyadewa P D J C
Supervisor	Mr. Dinith Primal
Co-Supervisor	Mr. Amila Nuwan
Date of submission	14 October 2026

B.Sc. (Hons) in Information Technology
Specializing in Cyber Security

1. Introduction

Large language models enable attackers to create fluent, personalized scam emails that may evade traditional filters. Project R26-CS-011 protects users of the Mozilla Thunderbird desktop email client using a lightweight system that runs locally. A Thunderbird Mail Extension sends each displayed email to a local Python backend, where an orchestrator coordinates three components through a validated JSON contract:

- Component 1 — Detection: a rule-based prefilter checks SPF, DKIM and DMARC alignment, look-alike domains, an offline cache of 226,073 malicious URLs and risky attachments. A fine-tuned DistilBERT scam classifier, an AI-authorship detector and a 0–100 risk scorer contribute to the result.
- Component 2 — Defense and Awareness: a deterministic policy engine produces defense recommendations and bounded awareness guidance. It also supports behavior profiling, a protected security log and a six-panel dashboard.
- Component 3 — Steganalysis: an EfficientNet-B2 model classifies image attachments as Clean or Stego, with Grad-CAM explanations, through the local Flask service StegoGuard.

The system aims to detect AI-driven scams, explain decisions and educate users without sending email content to the cloud. Since it processes private email and influences security decisions, secure coding and privacy were treated as design requirements. Data is minimized, interfaces are validated, and failed or incomplete analysis is never presented as safe.

2. Secure Coding Standards

The following standards were used as engineering guidance; the project does not claim formal certification.

2.1 OWASP Secure Coding Practices and API Security Top 10 (2023)

The OWASP Secure Coding Practices Quick Reference Guide informed the FastAPI services (C1 and C2), Flask service (C3), Thunderbird add-ons and dashboard. The OWASP API Security Top 10 was used to review local APIs.

Risk area	Project measures
API2 — Authentication	C2 bearer-token authentication uses constant-time comparison. Live dashboard updates use single-use tickets valid for 60 seconds.
API3 — Object/property authorization	Pydantic models reject unknown fields; the server assigns each record's data origin.
API4 — Resource consumption	Rate limit of 60 requests/minute before authentication, 256 KB request cap, bounded SSE subscribers and capacity-limited awareness workers.
API8 — Misconfiguration	Services bind to 127.0.0.1; public health output is minimal; detailed diagnostics require authentication; CSP and anti-clickjacking headers are used.
API10 — Unsafe API consumption	C2 revalidates C1/C3 envelopes, including finite confidence values and reference shapes, instead of trusting upstream components.

2.2 CERT Coding Standards

The CERT C/C++ standard does not directly apply because the project uses Python and JavaScript. Its underlying principles were adopted: validate input, heed lint warnings, use default-deny decisions and keep designs simple. Ruff enforces linting, while Bandit checks Python code for security issues mapped to CWE weaknesses, including SQL injection (CWE-89) and hard-coded credentials (CWE-798).

2.3 ISO/IEC 27001:2022 and supporting frameworks

Control / framework	Application
A.8.25 Secure development life cycle	Security activities are included in each SDLC phase.
A.8.26 Application security requirements	Privacy and security requirements include keeping email content on the user's machine.
A.8.28 Secure coding	OWASP practices, input validation, parameterized SQL and safe output encoding.
A.8.29 Security testing	pytest, Bandit, Ruff, audit probes and phase validators.
A.8.15 Logging; A.8.24 Cryptography	Privacy-safe audit logging, protected security logs, Fernet encryption and SHA-256 hashing.
A.5.34 Privacy and PII	Local processing, data minimization, and hashed or encrypted records.

Saltzer and Schroeder's principles, NIST SP 800-160 and the NIST Privacy Framework informed fail-safe defaults, least privilege and data handling. Email authentication follows IETF RFCs 7208 (SPF), 6376 (DKIM), 7489 (DMARC) and 5322. OWASP machine-learning supply-chain guidance informed model-file handling.

3. Secure Coding Techniques

3.1 Input validation

- C2: strict Pydantic contracts reject unknown fields, require finite confidence values between 0 and 1, validate reference identifiers and reject requests larger than 256 KB. SQLite statements are parameterized.
- C1: email data is parsed into typed schemas according to RFC 5322. URLs are normalized and checked for shorteners, data: URIs and mismatched link text. Malformed input produces a degraded verdict.
- C3: the Flask endpoint requires an image MIME type and decodes the file with Pillow before inference. The add-on sends only image/* attachments.
- Dashboard XSS prevention: untrusted text is inserted as text, not through innerHTML, and a Content Security Policy provides additional protection.

```
class C3Finding(BaseModel):\n    model_config = ConfigDict(extra="forbid")\n    label: Literal["clean", "stego"]\n    confidence: float = Field(ge=0.0, le=1.0, allow_inf_nan=False)
```

3.2 Error handling and fail-safe behavior

- C1 isolates pipeline stages; a stage exception marks the result degraded and identifies the failed stage rather than stopping the add-on.
- C2 treats a missing score as "analysis incomplete", never "clean". Awareness time-outs use deterministic guidance without changing the defense decision. Record responses distinguish new events (201), identical retries (200), conflicting duplicates (409) and recording failures (503).
- C3 returns an error for invalid images and fails gracefully when its backend is unavailable, so email reading is not disrupted (TC10). Detailed exceptions remain in logs rather than being exposed to users.

```
try:\n    result.stages["ai_generation"] = self.ai_detector.run(email)\nexcept Exception:\n logger.exception("AI-authorship stage failed")\n    result.degraded = True\n result.failed_stages.append("ai_generation")
```

3.3 Authentication, authorization and network exposure

- C2 requires a bearer token, compares it in constant time and applies a 60-requests/minute limit before authentication. Dashboard event tickets are single-use and expire after 60 seconds.
- Block and quarantine actions are protected by permission gates that are closed by default. Threat severity is separate from permission to act; optional machine learning can add awareness text but cannot change defense severity.
- C1 and C3 bind to 127.0.0.1. Thunderbird add-ons request only the message-reading, attachment and notification permissions needed.

```
def verify_token(supplied: str) -> bool:\n    return hmac.compare_digest(\n        supplied.encode(),
settings.API_TOKEN.encode()\n    )
```

3.4 Encryption, hashing and access control

- C2 can encrypt five sensitive log columns (detection category, evidence, awareness, warnings and errors JSON) using Fernet. The key comes from C2_LOG_KEY in the environment, not source code. Missing or incorrect keys fail closed.
- C1 stores a SHA-256 hash of the email body rather than the body itself. Analysis uses the local loopback interface; live DNS look-ups are disabled by default.
- Malicious URLs and attachments are not opened or executed. Model artifacts are checked against SHA-256 digests in a manifest and loaded from trusted project sources. Grad-CAM hooks are removed after requests; awareness workers are bounded and time-limited.
- Tests use synthetic tokens and disposable databases. The test harness refuses the live database path to prevent changes to real user history.

```
cur.execute(\n    "INSERT INTO detection_log (ts, risk_score, body_sha256, sender_domain) VALUES (?, ?, ?,
?)",\n    (now, score, hashlib.sha256(body.encode()).hexdigest(), registrable_domain),\n)
```

4. Tools and Technologies

Tool	Purpose	Recorded result
Bandit	Python static analysis for insecure patterns.	Zero findings (C2).
Ruff	Linting and quality checks.	Clean (C2).
pytest 8	Unit, integration and regression tests for parsers, scoring, logs and APIs.	C1: 38/38 passed; C2: 796 passed, 0 failed.
Phase validators	Scenario checks using disposable storage and before/after data-directory checks.	C2: 167/167; C1 phase checks passed.
Security audit probes	Authentication, oversized requests, rate limits and unknown-field checks.	C2: 15/15 passed.
Functional / end-to-end tests	Clean, stego, no-attachment and backend-down Thunderbird scenarios.	10 C3 test cases defined.
FastAPI, Pydantic, Fernet	Typed API contracts, schema generation and authenticated encryption.	Used in implementation.
Git and pinned requirements	Version control, frozen baselines and dependency-version control.	Versions pinned, including FastAPI 0.111.0 and Transformers 4.40.2.

Penetration-style testing used C2 audit probes and C1 parser/API robustness tests, including malformed emails and .eml files. OWASP ZAP dynamic scanning, pip-audit dependency scanning and a SonarQube quality gate remain planned before an external pilot.

5. Compliance Standards and Measures

5.1 Relevant standards and laws

- Sri Lanka Personal Data Protection Act, No. 9 of 2022 (PDPA): relevant because email addresses, names and message content can be personal data. The project's measures support purpose limitation, data minimization, retention limitation, confidentiality and transparency.
- General Data Protection Regulation (GDPR): used as an international benchmark for data minimization, purpose and storage limitation, privacy by design and default (Article 25), and security of processing (Article 32).
- HIPAA is not applicable because the system does not process health records. PCI DSS is not applicable because it does not process payment-card data.
- Software and dataset licenses: the project records permissive licenses for dependencies (including Apache 2.0 for Transformers and MIT for FastAPI); dataset and pretrained-model licenses must be checked before redistribution.

5.2 Measures implemented

Principle	Measure
Data minimization	C1 stores a subject preview limited to 200 characters, a SHA-256 body hash and sender's registrable domain—not the body or recipient. C2 receives structured findings, not raw email.
Purpose limitation	Data is used to analyze the opened email. User feedback does not automatically retrain models.
Integrity and confidentiality	Local processing, loopback-only services, C2 authentication, Fernet column encryption and fail-closed key handling.
Privacy by design/default	No cloud inference; live DNS look-ups are off by default; high-impact actions are closed by default.
Storage limitation	Logs can be disabled and C3 does not retain images after analysis. Retention periods must be set before a pilot.
Transparency	Scores show each stage's contribution; Grad-CAM heatmaps explain image classifications.

- Data protection: no third-party cloud service receives email content. Grad-CAM heatmaps may reproduce parts of an image and are treated as sensitive.
- Documentation: versioned interfaces (database schema 1.6, API schema 1.1, policy 1.5.0 and model 2.0), OpenAPI documentation, handover guidance, dataset provenance, hashed artifact manifests, backed-up threat-intelligence manifests and an AI-use disclosure are maintained.
- Consent: only public datasets were used and no real user email was collected. Users control automatic scanning, notification thresholds and audit logging. Any future user study will require documented informed consent.

6. Challenges and Mitigation Strategies

Challenge / current limitation	Mitigation and next step
Fernet encryption is implemented, but existing live history remained plaintext and key rotation was not available.	Encryption is opt-in with fail-closed key checks. Use copy-first migration, backup, transactions and WAL truncation; schedule key rotation and rehearse cut-over.
C1 and C3 local APIs lack authentication; loopback endpoints may still be called by other programs on the	Keep loopback binding and strict validation; extend C2's token scheme to C1/C3 or adopt secure inter-process communication.

same computer.	
Concurrent database requests caused SQLite lock contention.	Use WAL mode, short transactions and move blocking database work off the event loop; evaluate another repository backend.
PyTorch/pickle checkpoints may execute code when loaded.	Load only hash-verified trusted artifacts and move to weights-only or safetensors formats.
The AI-authorship detector flagged 20% of human-written emails.	Treat AI evidence as supporting only; hard-evidence floors (e.g. dangerous attachment = High) take precedence. Grad-CAM is attribution, not proof.
An early C2 audit found 25 issues; C3 kept one global “latest result”.	Fix findings with regression tests; per-message and per-attachment identifiers remain planned for C3.

Lessons and alternatives: security controls are easier to maintain when designed into shared contracts from the start. Future work should authenticate every component from day one, enable encryption by default with key rotation, and automate dependency and dynamic scanning in CI.

7. Secure SDLC, Code Review and Ethics

7.1 Secure Development Lifecycle

SDLC phase	Activities
Requirements	Define privacy/security NFRs: no email content leaves the machine; audit logs never store the body; AI evidence alone never marks an email malicious.
Design	Use local-only architecture, validated JSON contracts, separate threat severity from action permission, and closed-by-default gates.
Implementation	Apply OWASP practices, Pydantic validation, parameterized SQL, safe output encoding and environment-based secrets.
Testing	Run pytest, Bandit, Ruff, audit probes and phase validators with synthetic data and disposable databases.
Deployment and maintenance	Pin dependencies, run smoke tests, keep high-impact gates closed by default, and refresh threat intelligence with backups and manifests.

Security was emphasized especially during design, when the local-only architecture and fail-safe contract were selected, and testing, when boundaries such as missing data, incorrect keys, duplicate events and time-outs were exercised.

7.2 Code review and audit

Code review combined automated gates (Bandit, Ruff, pytest) with a full manual audit that reproduced each suspected bug against a scratch copy of the project, without modifying files or reading the live API token. The audit produced 25 findings: 1 critical, 3 high, 10 medium and 11 low, each recorded with location, impact, proposed fix and a verification step. Feedback was incorporated through a remediation plan of numbered phases, each targeting specific issue numbers (for example Phase 09 *API hardening* closed #6, #7, #11, #14 and #16). Every phase appended evidence to a progress log, and the fixes were converted into permanent regression checks in audit_probe.py and validate_all_phases.py, so a closed vulnerability cannot silently reappear. A subsequent independent evidence investigation re-executed ten test suites and recorded both confirmations and contradictions, which were fed back as open items in Section 6.

7.3 Ethical considerations

- Sensitive data: no private email from real users was collected; public datasets were used.
- Safe handling: malicious URLs and payloads were never opened or executed.

- Fairness: AI-authorship is not a verdict on its own because detectors can mislabel non-native English writers.
- User dignity: the behavior profiler summarizes interactions without labelling a user “vulnerable”.
- Honest reporting: limitations are disclosed; a steganographic image is not automatically described as malicious, and no model is presented as infallible.

8. References

[1] OWASP Foundation, “OWASP Secure Coding Practices — Quick Reference Guide.” https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/

[2] OWASP Foundation, “OWASP API Security Top 10 — 2023.” https://owasp.org/API-Security/

[3] OWASP Foundation, “OWASP Machine Learning Security Top 10.” https://owasp.org/www-project-machine-learning-security-top-10/

[4] Carnegie Mellon University SEI, “CERT Secure Coding Standards.” https://www.sei.cmu.edu/our-work/secure-development/

[5] MITRE, “Common Weakness Enumeration (CWE).” https://cwe.mitre.org/

[6] ISO/IEC 27001:2022, Information security, cybersecurity and privacy protection — Information security management systems — Requirements.

[7] Parliament of the Democratic Socialist Republic of Sri Lanka, Personal Data Protection Act, No. 9 of 2022.

[8] European Union, Regulation (EU) 2016/679 — General Data Protection Regulation. https://gdpr-info.eu/

[9] NIST, “NIST Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management, Version 1.0,” CSWP 10, 2020.

[10] NIST, “Engineering Trustworthy Secure Systems,” SP 800-160 Vol. 1 Rev. 1, 2022.

[11] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” Proc. IEEE, vol. 63, no. 9, pp. 1278–1308, 1975.

[12] IETF, RFC 7208 (SPF), RFC 6376 (DKIM), RFC 7489 (DMARC) and RFC 5322 (Internet Message Format).

[13] PyCQA, “Bandit — a security linter for Python.” https://bandit.readthedocs.io/

[14] Astral, “Ruff — Python linter.” https://docs.astral.sh/ruff/

[15] Python Cryptographic Authority, “Fernet (symmetric encryption).” https://cryptography.io/en/latest/fernet/

[16] FastAPI documentation, https://fastapi.tiangolo.com/ ; Pydantic documentation, https://docs.pydantic.dev/ ; pytest documentation, https://docs.pytest.org/

[17] Mozilla, “Thunderbird WebExtension (MailExtension) APIs.” https://webextension-api.thunderbird.net/